

3-Mark Questions & Answers -- Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain the concept of ADT with a suitable example.

An Abstract Data Type (ADT) is a high-level specification of a set of data items and the operations performed on them, completely independent of how the data is stored in memory or how operations are implemented.

* Example: List ADT

Q2. What are the advantages of using ADTs in programming

1. Modularity & Reusability: Software is broken into independent modules. An ADT implementation can be modified or optimized without altering client code.

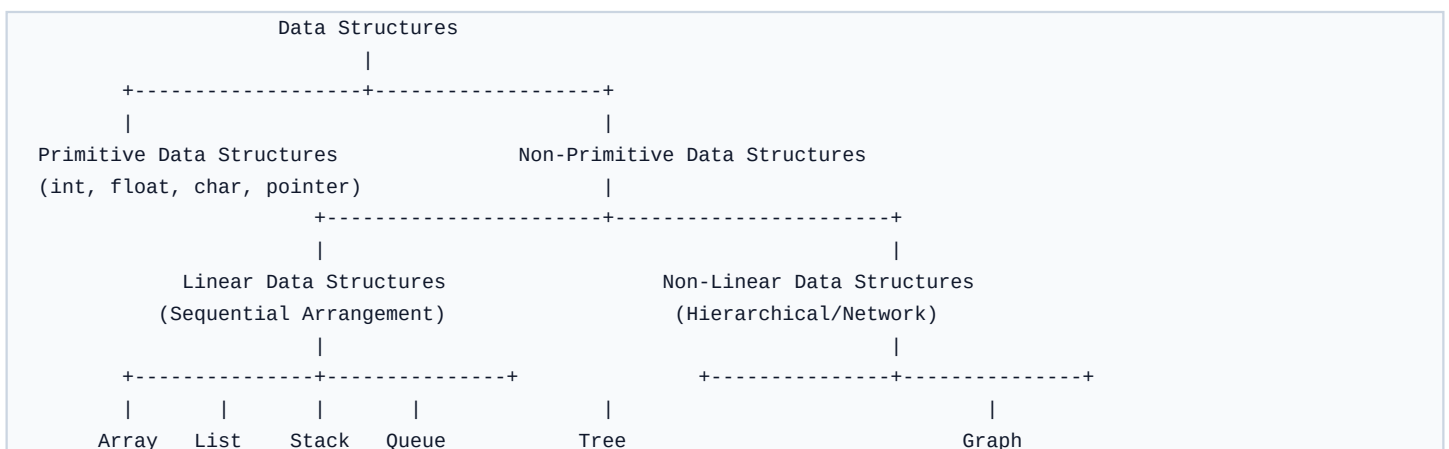
Q3. Explain the relationship between ADT and encapsulation.

* Encapsulation is the object-oriented programming mechanism of wrapping data and methods into a single unit (class/structure) and restricting direct access to private data.

Topic 2: Types of Data Structures

Q4. Classify data structures with a neat diagram.

Data structures are categorized into Primitive and Non-Primitive, with Non-Primitive further split into Linear and Non-Linear structures:



Q5. Explain primitive vs non-primitive data structures with examples.

Property	Primitive Data Structures	Non-Primitive Data Structures
Definition	Basic data types directly supported by	Complex structures formed by combining

Memory Size	Fixed size defined by language compile	Variable size dependent on element cou
Focus	Storing single basic values.	Organizing and managing groups of rela
Examples	`int`, `float`, `char`, `double`, `voi	Arrays, Linked Lists, Stacks, Queues,

Q6. Compare linear and non-linear data structures (any three points).

Feature	Linear Data Structures	Non-Linear Data Structures
Arrangement	Elements are arranged sequentially in	Elements are arranged hierarchically i
Traversal	Single run traversal can visit all ele	Requires complex tree/graph traversal
Memory Utilization	Can lead to inefficient memory use (e.	Efficient memory usage according to no
Examples	Array, Linked List, Stack, Queue.	Tree, Binary Search Tree, Graph.

Topic 3: Operations on Data Structures

Q7. Explain insertion and deletion operations with examples.

- * Insertion Operation: The process of adding a new data element into an existing data structure.

Q8. Explain sorting and merging operations on data structures.

- * Sorting Operation: Arranging elements of a data structure in a specific numerical or alphabetical order (Ascending or Descending).

Q9. Differentiate between traversal and searching operations.

Feature	Traversal Operation	Searching Operation
Objective	Visit every single element in the stru	Locate a specific target value.
Execution	Always inspects 100% of elements.	Stops immediately once target element
Output	Performs an action on all elements (e.	Returns index/location of target or -1

Topic 4: Arrays & Multi-Dimensional Arrays

Q10. Explain row-major and column-major order representation of a 2-D array.

A 2-D array is logically a matrix but stored in 1-D contiguous physical memory:

Q11. Derive the address calculation formula for an element in a 2-D array (row-major).

For a 2-D array $A[m][n]$ stored in Row-Major Order with Base Address B and element size w:

Equation:

(Assuming LBR = LBC = 0, Address = $B + w \times (i \times n + j)$).

Q12. Write a short note on sparse matrices.

A Sparse Matrix is a matrix in which the majority of elements are zero.

Topic 5: String Manipulation

Q13. Explain any three string manipulation functions with examples.

1. `strlen(s)`: Calculates length excluding null character.

Q14. Write a note on pattern matching in strings.

Pattern Matching is the process of checking whether a specific substring (called Pattern P of length m) exists inside a larger text string (called Text T of length n).

Q15. Explain the concept of string representation as an array of characters.

In languages like C, strings are represented as 1-D arrays of char primitive elements.

Topic 6: Self-Referential Structures with Pointers

Q16. Explain self-referential structures with a suitable example and diagram.

A Self-Referential Structure contains a pointer variable pointing to another structure of the same type.

Diagram:

Q17. Explain how self-referential structures are used to implement linked lists.

Dynamic linked lists cannot use static contiguous arrays. Instead:

Q18. Differentiate between arrays and self-referential (pointer-based) structures.

Feature	Arrays	Self-Referential Structures
Memory Allocation	Contiguous memory allocated at compile	Non-contiguous memory allocated dynami
Insertion / Deletion	Slow $O(n)$ due to element shifting.	Fast $O(1)$ by updating pointer link
Memory Overhead	Zero pointer overhead.	Extra pointer size (4 or 8 bytes per n